Zenith Bot v022 - ■■■■■■■■ ■■■■■■■ ■■■■■ ■ ■■■■■■■■

■ LIVE BUILD 7b62294 - SIMPLE TELEGRAM v022

■■■■ ■■■■:
- 687 ■■ ■■ ■■■■■■■
- 8 ■■■■■ ■■■■■ (sleeves, test, file, health, tick, start/stop...)
- ■■■■■■ ■■■■ ■■■■■■■ ■■■■■■■■ ■■■■ ■■■■■■■■
- polling ■■■■■■■■■

■■■ ■■ ■■■■■ ■ ■■■■■■■■:
- 429 ■■ (38% ■■■■■)
- ■■■■ 4 ■■■■■■ ■■■■■:
■ ■■■■■■ - ■■■■■■■■ ■■■■■ ■■■■■■■■■■ ■■■■■■■■■■
■ ■■■■■■■■■■ - ■■■■■ ■■■■■ ■■ PnL
■ ■■■■■■■■ - ■■■■■■■■ ■ ■■■ ■■■■■■
■ ■■■■■■■■ - 10 ■■■■■■■ ■■■■■
- ■■■■■ ■■■■■ ■■■■ ■■■■■ ■■■■■■■ ■■■■■ ■■■■■ (■■ ■■■■■■)
- ■■■■■■■■ ■■■■■ ■■■■■: /close BTC, /closeall, /status, /panel
- polling ■■■■■■■ ■■ 409 handling
- ■■■■■ ■■■■■■■■ sleeves/test/file

Health Live:
- build: 7b62294 ■
- assets: ['BTC','ETH'] ■ (■■■ 2)
- capital: 400$ lev1 rebalance25 loop90 ■
- orders: ■■■ 2 ■■■■■■ BTC,ETH (■■ 10) ■
- telegram: enabled true, poll true, sent 1 ■

■■■■■■■■ ■■■■■■■:
/start /panel /help - ■■■■ ■■■■■ + ■■■■■■■
/status - ■■■■■■ ■■■■
/positions - ■■■■■■■■■■■■ ■■■■■
/balance - ■■■■■■■
/trades - ■■■■■■■■ ■■■■■
/close BTC - ■■■■■ ■■ ■■■■■■■
/closeall - ■■■■■ ■■■■

■■■■■■■ ■■■■■:
[■ ■■■■■■] [■ ■■■■■■■■■■]
[■ ■■■■■■■] [■ ■■■■■■■■]
[■ BTC] [■ ETH] (■■■■ ■■■■ ■■■■■)
[■ ■■■■■ ■■■■] (■■■■ ■■■■ ■■■■■■■ ■■■■■)
[■ ■■■■ ■■■■■]
[■ ■■■■■■■■ ■■]

■■■■■■■■ 21/21 ■■■■■■■ ■

■■■■: app/telegram_bot.py | 18683 ■■■■■■■ | 429 ■■ - SIMPLE v022

```
"""
Zenith Trader - Telegram Simple Practical Dashboard v022
■■■■■■ ■■■■■■■■■ ■■■■■ ■■■■■■■
- ■■■■ ■■■■■■■■ ■■■■■■
- ■■■■ ■■■■■ ■■ ■■■■■■■■■ ■■■■■■■■
- ■■■■■ ■■■■■■■■ sleeves/test/file
- polling ■■■■■■■
"""
from __future__ import annotations
import logging
import threading
import time
from datetime import datetime, timezone
from typing import List
import httpx

log = logging.getLogger("telegram_simple")

def _now():
    return datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")

class TelegramReporter:
    def __init__(self, bot_token: str, chat_id: str, dashboard_url: str = ""):
        self.bot_token = bot_token
        self.chat_id = chat_id
        self.dashboard_url = dashboard_url
        self._engine = None
        self._stop = threading.Event()
        self._poll_thread = None
        self._offset = 0
        self.enabled = bool(bot_token and chat_id)
        self.poll_commands = True
        self.stats = {"sent": 0, "failed": 0}
        self.last_error = None
        self.version = "v022-SIMPLE-PRACTICAL-BTC-ETH"

    def bind_engine(self, engine):
        self._engine = engine

    def _api(self, method: str, **params):
        if not self.enabled:
            return {}
        try:
            url = f"https://api.telegram.org/bot{self.bot_token}/{method}"
            clean = {k: v for k, v in params.items() if v is not None}
            resp = httpx.post(url, json=clean, timeout=20)
            try:
                data = resp.json()
            except:
                data = {}
            if resp.status_code == 200 and data.get("ok"):
                return data
            else:
                log.warning(f"Telegram {method} fail {resp.status_code}: {str(data)[:300]}")
                self.last_error = str(data)[:300]
                self.stats["failed"] += 1
                return data
        except Exception as e:
            log.warning(f"Telegram {method} error: {e}")
            self.last_error = str(e)[:300]
            self.stats["failed"] += 1
            return {}

    def send(self, text: str, reply_markup: dict = None):
        if not self.enabled:
            return False
        try:
            # Telegram limit 4096 chars
            if len(text) > 4000:
                text = text[:4000] + "\n... (truncated)"
            data = self._api("sendMessage", chat_id=self.chat_id, text=text, parse_mode="HTML", reply_markup=reply_markup, disable_web_page_preview=Tr
            ok = bool(data.get("ok"))
            if ok:
                self.stats["sent"] += 1
            return ok
        except Exception as e:
            log.warning(f"send failed: {e}")
            return False

    def simple_keyboard(self, positions: List[dict] = None) -> dict:
        # ■■■■■ ■ ■■■■■■■■: 3 ■■■■■ ■■■■■
        keyboard = [
            [{"text": "■ ■■■■■■", "callback_data": "status"}, {"text": "■ ■■■■■■■■■■", "callback_data": "positions"}],
            [{"text": "■ ■■■■■■■", "callback_data": "balance"}, {"text": "■ ■■■■■■■■", "callback_data": "trades"}],
        ]
        # ■■■■■■■■■ ■■■■■ ■■■ ■■■ ■■■■■■■ ■■■■■ ■■■■■
        if positions:
            active = [p for p in positions if abs(float(p.get("size", 0) or p.get("size_currency", 0) or 0)) > 1e-9]
            if active:
                close_row = []
                for p in active[:4]:
                    # ■■■■■■■■ asset ■■ instrument
                    inst = p.get("instrument_name", "") or p.get("instrument", "")
                    asset = inst.replace("USDT", "").replace("USD", "").replace("/", "")[:6] or "?"
                    close_row.append({"text": f"■ {asset}", "callback_data": f"close:{asset}"})
                    if len(close_row) == 2:
                        keyboard.append(close_row)
                        close_row = []
                if close_row:
                    keyboard.append(close_row)
                keyboard.append([{"text": "■ ■■■■■ ■■■■", "callback_data": "closeall"}])

        keyboard.append([{"text": "■ ■■■■ ■■■■■", "callback_data": "panel"}])
        if self.dashboard_url:
            keyboard.append([{"text": "■ ■■■■■■■■ ■■", "url": self.dashboard_url}])
        return {"inline_keyboard": keyboard}

    def format_status(self, snap: dict) -> str:
        try:
            cfg = snap.get("config", {})
            risk = snap.get("risk", {})
```

```
        bot = snap.get("bot", "zenith")
        cap = cfg.get("capital_usd", 400)
        assets = cfg.get("assets", ["BTC","ETH"])
        lev = cfg.get("lev_cap", 1)
        loop = cfg.get("loop_seconds", 90)
        version = cfg.get("version", self.version)
        est = risk.get("estimated_bot_equity", cap)
        dd = risk.get("drawdown_pct", 0) * 100
        peak = risk.get("peak_bot_equity", cap)
        pos_count = len([p for p in snap.get("positions", []) if abs(float(p.get("size",0) or 0))>1e-9])
        orders = len(snap.get("orders_log", []))
        last_fill = snap.get("last_fill_at", "—")[:19] if snap.get("last_fill_at") else "—"

        txt = f"<b>■ Zenith Bot - ■■■■■</b>\n"
        txt += f"■■■■■■■■■■■■■■■■\n"
        txt += f"■ ■■■■: {version}\n"
        txt += f"■ ■■■■■■■: ${cap} | ■■■■: x{lev}\n"
        txt += f"■ ■■■■■■■■■■: {', '.join(assets)} ({len(assets)})\n"
        txt += f"■ ■■■: {loop}s | ■■■■■■■ ■■■■: {pos_count}\n"
        txt += f"■ ■■■ ■■■■■■■: ${est:.2f}\n"
        txt += f"■ ■■■■■■■■: {dd:.2f}% | ■■■: ${peak:.2f}\n"
        txt += f"■ ■■■■■■■■: {orders} | ■■■■■: {last_fill}\n"
        txt += f"■ {_now()}\n"
        txt += f"■■■■■■■■■■■■■■■■\n"
        txt += f"■ ■■■■■ ■■■■ ■ ■■ ■■■ ■■■■■■■"
        return txt
    except Exception as e:
        return f"■ ■■■■■: ■■■ {e}\n{_now()}"

def format_positions(self, snap: dict) -> str:
    try:
        positions = snap.get("positions", [])
        active = [p for p in positions if abs(float(p.get("size",0) or p.get("size_currency",0) or 0))>1e-9]
        if not active:
            return f"<b>■ ■■■■■■■■■■</b>\n■■■■■■■■■■■■■■■■\n■ ■■■ ■■■■■■■ ■■■■■■ ■■■■■\n■ {_now()}"
        txt = f"<b>■ ■■■■■■■■■■■ ■■■■ ({len(active)})</b>\n■■■■■■■■■■■■■■■■\n"
        for p in active[:10]:
            inst = p.get("instrument_name", "?")
            asset = inst.replace("USDT","").replace("USD","")[:6]
            size = float(p.get("size",0) or p.get("size_currency",0) or 0)
            direction = p.get("direction", "long" if size>0 else "short")
            icon = "■" if direction in ("buy","long") or size>0 else "■"
            entry = p.get("average_price", p.get("entry_price", 0))
            mark = p.get("mark_price", p.get("current_price", 0))
            pnl = p.get("unrealized_pnl", 0)
            try:
                txt += f"{icon} <b>{asset}</b> {direction.upper()} {size:.4f}\n"
                txt += f"   ■■■■: {float(entry):.2f} | ■■■■: {float(mark):.2f}\n"
                txt += f"   PnL: {float(pnl):+.2f} $\n"
            except:
                txt += f"{icon} {asset} {size}\n"
        txt += f"■■■■■■■■■■■■■■■■\n■ {_now()}"
        return txt
    except Exception as e:
        return f"■ ■■■■■■■: ■■■ {e}"

def format_balance(self, snap: dict) -> str:
    try:
        risk = snap.get("risk", {})
        cfg = snap.get("config", {})
        cap = cfg.get("capital_usd", 400)
        est = risk.get("estimated_bot_equity", cap)
        avail = risk.get("available_funds", cap)
        peak = risk.get("peak_bot_equity", cap)
        dd = risk.get("drawdown_pct", 0)*100

        txt = f"<b>■ ■■■■■■■ ■ ■■■■■■■</b>\n"
        txt += f"■■■■■■■■■■■■■■■■\n"
        txt += f"■ ■■■■■■■ ■■■■■■: ${cap}\n"
        txt += f"■ ■■■■■■■ ■■■■■■■: ${est:.2f}\n"
        txt += f"■ ■■ ■■■■■■: ${avail:.2f}\n"
        txt += f"■ ■■■■: ${peak:.2f}\n"
        txt += f"■ ■■■■■■■■: {dd:.2f}% (■■■: 15%)\n"
        txt += f"■■■■■■■■■■■■■■■■\n"
        txt += f"■■ ■■■■■: ■■■■■■■■■■■■■■■\n"
        txt += f"■ {_now()}"
        return txt
    except Exception as e:
        return f"■ ■■■■■■■: ■■■ {e}"

def format_trades(self, snap: dict) -> str:
    try:
        orders = snap.get("orders_log", [])[-10:]
        if not orders:
            return f"<b>■ ■■■■■■■■ ■■■■■</b>\n■■■■■■■■■■■■■■■■\n■ ■■■■■■■■■■ ■■■■ ■■■■■\n■ {_now()}"
        txt = f"<b>■ 10 ■■■■■■■ ■■■■■</b>\n■■■■■■■■■■■■■■■■\n"
        for o in reversed(orders):
            asset = o.get("asset","?")
            direction = o.get("direction","?")
            amount = o.get("amount",0)
            price = o.get("price",0)
            status = o.get("status","?")
            icon = "■" if direction=="buy" else "■"
            try:
                txt += f"{icon} {asset} {direction} {float(amount):.4f} @ {float(price):.2f} [{status}]\n"
            except:
                txt += f"{icon} {asset} {direction} {amount} @ {price}\n"
        txt += f"■■■■■■■■■■■■■■■■\n■ {_now()}"
        return txt
    except Exception as e:
        return f"■ ■■■■■■■■: ■■■ {e}"

def send_panel(self):
    snap = {}
    try:
        if self._engine:
            snap = self._engine.snapshot()
    except:
        pass
    pos = snap.get("positions", []) if snap else []
    text = self.format_status(snap)
    kb = self.simple_keyboard(pos)
    self.send(text, reply_markup=kb)
```

```
def handle_callback(self, cq: dict):
    try:
        data = cq.get("data","")
        cb_id = cq.get("id","")
        # Answer callback to remove loading
        try:
            self._api("answerCallbackQuery", callback_query_id=cb_id, text="■■ ■■■ ■■■■■■...")
        except:
            pass

        eng = self._engine
        snap = eng.snapshot() if eng else {}

        if data == "panel" or data == "home":
            self.send_panel()
        elif data == "status":
            self.send(self.format_status(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
        elif data == "positions":
            self.send(self.format_positions(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
        elif data == "balance":
            self.send(self.format_balance(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
        elif data == "trades":
            self.send(self.format_trades(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
        elif data.startswith("close:"):
            asset = data.split(":")[1].upper()
            self.send(f"■ ■■ ■■■ ■■■■ {asset}...")
            if eng:
                try:
                    result = eng.close_position(asset)
                    self.send(f"■ ■■■■■■■ ■■■■ {asset} ■■■■■ ■■\n■■■■■: {result}", reply_markup=self.simple_keyboard())
                except Exception as e:
                    self.send(f"■ ■■■ ■■■■ {asset}: {e}", reply_markup=self.simple_keyboard())
        elif data == "closeall":
            self.send("■ ■■ ■■■ ■■■■ ■■■ ■■■■■■■■■...")
            if eng:
                try:
                    for a in eng.settings.effective_assets:
                        try:
                            eng.close_position(a)
                        except:
                            pass
                    self.send("■ ■■■■■■■ ■■■■ ■■■ ■■■■■ ■■", reply_markup=self.simple_keyboard())
                except Exception as e:
                    self.send(f"■ ■■■: {e}", reply_markup=self.simple_keyboard())
        else:
            self.send_panel()
    except Exception as e:
        log.exception(f"callback error {e}")
        self.send(f"■ ■■■: {e}")

def _poll_loop(self):
    log.info("telegram SIMPLE polling started v022")
    while not self._stop.is_set():
        try:
            data = self._api("getUpdates", offset=self._offset, timeout=20, allowed_updates=["message","callback_query"])
            for upd in data.get("result") or []:
                self._offset = max(self._offset, int(upd.get("update_id",0))+1)
                if "callback_query" in upd:
                    self.handle_callback(upd["callback_query"])
                    continue
                msg = upd.get("message") or {}
                chat = msg.get("chat") or {}
                if str(chat.get("id")) != str(self.chat_id):
                    continue
                text = (msg.get("text") or "").strip()
                if not text:
                    continue
                # ■■■■■■■ ■■■■
                cmd = text.split()[0].lower().split("@")[0]
                arg = " ".join(text.split()[1:]) if len(text.split())>1 else ""
                self._on_command(cmd, arg)
        except Exception as e:
            log.warning(f"poll error: {e}")
            time.sleep(3)
        time.sleep(0.5)
def _on_command(self, cmd: str, arg: str = ""):
    eng = self._engine
    snap = eng.snapshot() if eng else {}
    if cmd in ("/start","/panel","/help"):
        help_text = (
            "<b>■ Zenith Bot - ■■■■■■</b>\n"
            "■■■■■■■■■■■■■■■\n"
            "■ /status - ■■■■■ ■■■\n"
            "■ /positions - ■■■■■■■■■■ ■■■■\n"
            "■ /balance - ■■■■■■■\n"
            "■ /trades - ■■■■■■■■ ■■■■\n"
            "■ /close BTC - ■■■■■ ■■■■■■■\n"
            "■ /closeall - ■■■■■ ■■■\n"
            "■ /panel - ■■■ ■■■■\n"
            "■■■■■■■■■■■■■■■\n"
            f"■■■■: {self.version}\n"
            f"■■■■■■: BTC,ETH | ■■■■■■: $400 | ■■■■: x1\n"
            f"■ {_now()}"
        )
        self.send(help_text, reply_markup=self.simple_keyboard(snap.get("positions",[])))
    elif cmd == "/status":
        self.send(self.format_status(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
    elif cmd == "/positions":
        self.send(self.format_positions(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
    elif cmd == "/balance":
        self.send(self.format_balance(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
    elif cmd == "/trades":
        self.send(self.format_trades(snap), reply_markup=self.simple_keyboard(snap.get("positions",[])))
    elif cmd == "/close":
        if not arg:
            self.send("■■■■■■■■: /close BTC ■■ /close all", reply_markup=self.simple_keyboard(snap.get("positions",[])))
            return
        asset = arg.strip().upper()
        if asset == "ALL":
            self.send("■ ■■■■ ■■■...")
            if eng:
                for a in eng.settings.effective_assets:
                    try: eng.close_position(a)
```

```
                        except: pass
                    self.send("■ ■■■ ■■■■ ■■", reply_markup=self.simple_keyboard())
            else:
                self.send(f"■ ■■■■ {asset}...")
                if eng:
                    try:
                        res = eng.close_position(asset)
                        self.send(f"■ {asset} ■■■■ ■■: {res}", reply_markup=self.simple_keyboard())
                    except Exception as e:
                        self.send(f"■ ■■■ {asset}: {e}", reply_markup=self.simple_keyboard())
        elif cmd == "/closeall":
            self.send("■ ■■■■ ■■■...")
            if eng:
                for a in eng.settings.effective_assets:
                    try: eng.close_position(a)
                    except: pass
                self.send("■ ■■■■■■■ ■■■■■ ■■", reply_markup=self.simple_keyboard())
        else:
            self.send_panel()

    def start_background(self):
        if not self.enabled:
            log.warning("Telegram disabled - no token/chat")
            return
        if self._poll_thread and self._poll_thread.is_alive():
            return
        self._stop.clear()
        self._poll_thread = threading.Thread(target=self._poll_loop, daemon=True, name="tg-simple")
        self._poll_thread.start()
        log.info(f"Telegram SIMPLE started {self.version} chat={self.chat_id}")
        # ■■■ ■■■■■ ■■ ■■■■■
        def send_initial():
            time.sleep(3)
            try:
                self.send_panel()
            except Exception as e:
                log.warning(f"initial panel fail: {e}")
        threading.Thread(target=send_initial, daemon=True).start()

    def stop(self):
        self._stop.set()

    # ■■■■ ■■■■■■■ ■■ engine - ■■■■■ ■■■■■ ■■■■■■
    def send_trade_report(self, action: dict) -> bool:
        try:
            # ■■■■■■■ ■■ ■■■■ ■■■■■■■ ■■■■■
            try:
                from app.reporting.trade_reporter import format_single_trade
                text = format_single_trade(action, version=self.version)
            except:
                # fallback ■■■■
                asset = action.get("asset","?")
                direction = action.get("direction","?")
                amount = action.get("amount",0)
                price = action.get("price",0)
                icon = "■" if direction=="buy" else "■"
                text = f"{icon} <b>{asset} {direction}</b> {amount} @ {price}\n■ {action.get('order_id','?')}\n■ {_now()}"
            # ■■■■ ■■■■■■ ■■■■ ■■■■■ ■■■■■■ (■■■■ ■■■■■■■■)
            return self.send(text)
        except Exception as e:
            log.warning(f"trade report fail: {e}")
            return False

_reporter = None

def init_reporter(engine=None):
    global _reporter
    try:
        if engine and hasattr(engine, 'settings'):
            s = engine.settings
        else:
            from app.config import get_settings
            s = get_settings()
        if not s.telegram_enabled or not s.telegram_bot_token or not s.telegram_chat_id:
            log.warning(f"Telegram disabled: enabled={s.telegram_enabled} token={bool(s.telegram_bot_token)} chat={bool(s.telegram_chat_id)}")
            return None
        _reporter = TelegramReporter(
            bot_token=s.telegram_bot_token,
            chat_id=s.telegram_chat_id,
            dashboard_url=getattr(s, 'dashboard_url', '') or "",
        )
        if engine:
            _reporter.bind_engine(engine)
        return _reporter
    except Exception as e:
        log.warning(f"init_reporter fail: {e}")
        return None

def get_reporter():
    return _reporter
```